Sim-to-Real RL for Robust LiDAR-Based Navigation

Simulation-phase pipeline, experiment design and 7-day plan

Project	Mapless goal navigation with a PPO policy on a TurtleBot3 Burger; effect of sensor and dynamics domain randomization
Owners	**Saigirish S**: Gazebo/Gymnasium environment, PPO training, reward, randomization, simulation benchmark. **Siddhi Vinayak**: baseline controller, evaluation harness, ROS 2 deployment node, hardware setup and analysis.
Status	Simulation phase first (7 days). Hardware phase starts only after the Day 7 readiness checklist (Section 10).

1. Objective, scope and claims

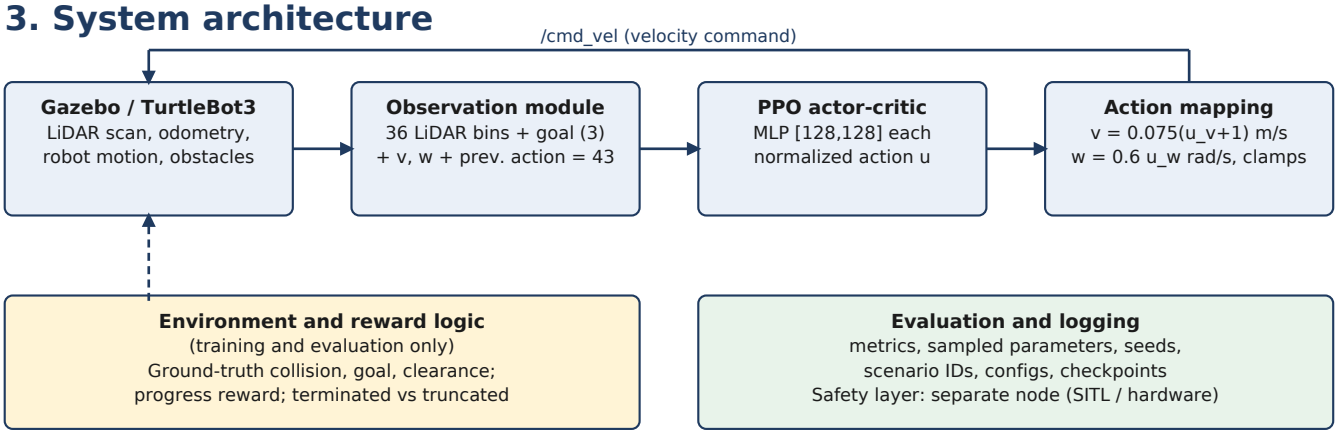
Research question. Does domain randomization of sensor observations and robot dynamics improve the robustness of a PPO-based LiDAR navigation policy beyond nominal-simulation training, and what does it cost under nominal conditions?

- **Contribution:** a controlled 2×2 comparison of four policies (P1-P4). Comparing RL algorithms is not a goal.
- **Out of scope:** SLAM, map-based navigation, multi-robot work, recurrent policies, model-based RL, other algorithms (SAC and RecurrentPPO are future work). Nav2 is only an optional *privileged* reference, since it uses a map and localization.
- **Claim policy:** simulation results support claims about robustness to *simulated* perturbations only. Sim-to-real claims need hardware trials with external ground truth. Confirm on Day 1 whether the robot is available and which LiDAR it has (LDS-01 or LDS-02; they differ in range, accuracy and beam layout).

2. Frozen design decisions

Item	Decision	Status
Algorithm	SB3 PPO, `MlpPolicy`, continuous actions; separate actor and critic MLPs, 2×128 hidden units, tanh	Fixed
Control period	Equal to the LiDAR scan period (0.2 s if the scan rate is 5 Hz); one action per new scan, held until the next	Verify Day 1
Observation	43 values: 36 LiDAR bins, goal distance, sin and cos of goal bearing, v, ω, previous action (2)	Fixed
Action	u in $[-1, 1]^2$; v = 0.075(u_v+1) m/s (0 to 0.15); ω = 0.6 u_ω rad/s (±0.6)	Fixed; limits provisional
Arena and task	4 m × 4 m square (match the physical arena if known); 4-8 static obstacles (boxes 0.2-0.4 m, cylinders r = 0.1-0.2 m); start-goal distance 1.5-3.5 m; minimum spawn clearance 0.4 m	Provisional
Goal tolerance	0.20 m	Provisional
Timeout	350 steps (70 s at 0.2 s); recheck against hand-coded controller completion times	Provisional
Collision	Ground-truth robot footprint (URDF collision geometry) against obstacle and wall geometry. Never LiDAR minimum range	Fixed
Motion distortion	Per-beam scan skew from simulated robot motion, applied in the shared simulator wrapper for **all** policies (nominal sensor fidelity, not randomization)	Fixed
Checkpoint rule	Final checkpoint of every run. The validation set monitors convergence only; no checkpoint selection	Fixed
Seeds	3 per policy (fallback: 2); trained seed-major (Section 7)	Fixed
Safety layer	Independent of the policy; active in SITL and hardware only, not in training	Fixed

3. System architecture



- Training uses the Gymnasium environment. The later ROS 2 deployment node imports the **same** observation-preprocessing module.
- Ground truth feeds the reward, termination and evaluation only; it is never part of the observation.
- Gazebo Classic control (ROS 2 Humble): `/pause_physics` and `/unpause_physics` come from the `gazebo_ros_init` plugin; `/set_entity_state` needs the `gazebo_ros_state` plugin.

4. Observation, action and estimated-pose model

Component	Dim	Definition	Normalization
LiDAR bins	36	Min-pooled 10° bins; bin 0 centred on forward; invalid readings (0, inf, NaN) mapped to max range	range / r_{max}, clip [0, 1]
Goal distance	1	From the estimated pose	d / arena diagonal, clip [0, 1]
Goal bearing	2	sin and cos of the estimated bearing	none
Linear velocity	1	Measured: odom twist through the measurement model	v / 0.15
Angular velocity	1	Measured: same source	ω / 0.6
Previous action	2	Last normalized u; zeros at reset; same convention in deployment	none
Total	**43**		

- **Estimated pose.** Integrate the measured v and ω from the reset pose; the goal lives in the reset odometry frame; goal distance and bearing come from this pose. P1 uses the error-free measurement model. Never add independent noise to goal distance and bearing per step: real drift accumulates.
- **Dropout vs pooling.** Single-beam dropout rarely blanks a whole 10° bin after min-pooling, so P2 uses block dropout, and a unit test must show that it changes the binned observation.
- **Day 1 metadata audit.** Record `len(ranges)`, angle_min/max/increment, range_min/max, scan_time, frame_id, invalid encoding and QoS for the simulated scan (and the real one if available) in `configs/env.yaml`.

5. Environment, reward and reset logic

Step loop

1) wait for a scan strictly newer than the last; 2) build the observation; 3) apply the action with zero-order hold for one control period; 4) advance the simulation; 5) compute reward and end conditions. Use simulation time throughout.

Episode endings (Gymnasium semantics)

- **terminated**: goal reached (ground-truth distance ≤ 0.20 m) or collision (footprint contact).
- **truncated**: timeout only. Unit-test that the pinned SB3 version bootstraps truncated episodes correctly.

Reset sequence

Stop the robot with explicit zero twist; set a valid pose with a unit quaternion (fill every field); teleport the pre-spawned obstacle pool and park unused obstacles outside the arena; reject overlaps; set the goal in the reset odometry frame; wait for a scan newer than the reset time; clear the previous-action and delay buffers.

Reward (identical for P1-P4)

```
r_t = k (d_t - d_t+1) - c_step + R_goal*[goal] - R_col*[collision] - r_clear - r_smooth
```

d = **ground-truth** goal distance, evaluated on every transition including terminal ones. Start values: k = 10, c_step = 0.05, R_goal = 100, R_col = 100, r_clear = 0, r_smooth = 0. Log each component separately.

- Do **not** use the form k[γΦ(s') - Φ(s)] with zero terminal potential: a collision far from the goal would receive positive shaping.
- **Clearance decision (P1 pilot, then freeze):** if minimum ground-truth clearance is regularly below 0.05 m, add r_clear = 0.5 (0.20 - c)/0.20 for clearance c < 0.20 m, or inflate the training footprint by 0.03 m.
- **Smoothness decision (P1 pilot, then freeze):** if mean |Δu| exceeds about twice the hand-coded controller's, or commands are bang-bang, add r_smooth = 0.05 $\|u_t - u_{t-1}\|^2$.
- **Reward audit:** stationary, random, spin-in-place and hand-coded traces; the hand-coded controller must clearly score highest.

Hand-coded reactive baseline (same 43-value observation)

Turn proportionally to the goal bearing and scale forward speed by cos(bearing error). If the minimum range in the ±30° front sector is below 0.5 m, slow down and turn toward the side with the larger mean range; below 0.25 m, stop forward motion and rotate. It is both the environment-correctness test and the reported classical baseline.

6. Experimental design: P1-P4

Policy	Sensor / observation randomization	Dynamics randomization
P1 Nominal	Off	Off
P2 Sensor	On	Off
P3 Dynamics	Off	On
P4 Combined	On (same marginal ranges as P2)	On (same marginal ranges as P3)

Held identical across P1-P4: algorithm, network, hyperparameters, reward, observation, action mapping, arena and task distribution, layout random-number stream, step budget, evaluation sets. Start, goal and layout variation is task variation, not domain randomization.

Provisional randomization ranges

Starting values only. Replace them with measured values if robot characterization is possible (static scans at several distances, step responses, odometry error). Sample once per episode unless stated. Freeze before the main runs.

Group	Parameter	Range	Sampling
P2	LiDAR range noise σ = a + b·r	a ~ U(0.003, 0.015) m; b ~ U(0, 0.01)	per beam, per scan
P2	Random beam dropout	p ~ U(0, 0.05)	per beam, per scan
P2	Block dropout (invalid regions)	prob. ~ U(0, 0.3) per scan; 1-2 blocks of 3-10 beams	per scan
P2	Sensor delivery latency	0 or 1 extra scan (0-0.2 s)	episode
P2	LiDAR mounting offset	yaw ±3°; x, y ±0.02 m	episode
P2	Odometry scale error	±5% on measured v (and integrated pose)	episode
P2	Odometry yaw-rate bias	±0.005 rad/s	episode
P2	Velocity measurement noise	σ_v ~ U(0, 0.005) m/s; σ_ω ~ U(0, 0.02) rad/s	per step
P3	Command gain (v and ω separately)	× U(0.85, 1.15)	episode
P3	Actuator lag (first order)	τ ~ U(0, 0.3) s	episode
P3	Control delay	d ~ U(0, 0.2) s (previous action held for the first d seconds of the step)	episode
P3	Acceleration limits	linear U(0.2, 0.6) m/s^2; angular U(1, 3) rad/s^2	episode
P3	Friction and mass	friction × U(0.7, 1.3); mass × U(0.9, 1.2)	episode

- **Excluded from P3 by design:** wheel radius, because it also changes odometry. Its measurement-side effect is the odometry scale error in P2.
- **P4 constraint:** reject draws where sensor latency + control delay exceed 0.4 s. Check corner combinations with the hand-coded controller.
- **Isolation tests (required):** log all sampled parameters; P1 has every randomizer disabled; P2 changes no dynamics parameter; P3 changes no sensor or measurement parameter; P4 ranges equal the P2 and P3 marginals; hand-coded success at corner draws does not collapse (heuristic: at least 80% of its nominal success).

7. PPO training configuration and compute

Setting	Start value	Setting	Start value
Learning rate	3e-4 (constant)	GAE λ	0.95
Rollout length	2048 steps per env	Clip range	0.2
Minibatch size	256	Entropy coef. / value coef.	0.0 / 0.5
PPO epochs	10	Max grad norm	0.5
Discount γ	0.99	Initial log std	-0.5
Device	CPU; limit torch threads to about 2	Observation scaling	Fixed manual (no running stats)
Reward scaling	VecNormalize(norm_obs=False, norm_reward=True); save stats with checkpoints	Budget	Set from benchmark; provisional 1M steps per run

- **Budget rule:** required aggregate throughput = runs × steps per run / available seconds. Example: 12 runs × 1M steps in 48 h needs about 70 steps/s; 8 runs needs about 46 steps/s. If the benchmark is lower: drop seed 2 first, then cut steps per run equally for all policies (not below about 500k).
- **Workers:** benchmark 1, 2 and 4 headless Gazebo instances (more only if cores allow). Each gets its own `GAZEBO_MASTER_URI` port and `ROS_DOMAIN_ID`. Record steps/s, real-time factor, CPU, RAM and memory growth. The GPU does not speed up Gazebo physics.
- **Queue (seed-major):** P1 s0, P2 s0, P3 s0, P4 s0, then seed 1, then seed 2, so partial results stay balanced. Avoid heavy jobs during training.
- **Per run:** save config YAML, git hash, `pip freeze`, seeds, TensorBoard logs; checkpoint every 50k steps; evaluate 20 validation scenarios every 100k steps.
- **Recovery:** a watchdog restarts gzserver if no scan arrives for 2 s; training resumes from the last checkpoint. Test kill-and-resume before the main runs.

8. Evaluation protocol

Set	Size	Purpose
Train	Procedural stream	Training only
Validation	50 fixed scenarios	Convergence monitoring and pilot decisions
Test	100 fixed scenarios + pre-generated perturbation draws shared by all policies	Reported results; paired comparisons
Stress	About 10 (corridor, 0.5 m gap, dead end)	Reported separately, never tuned on

- **Seeds and files:** train, validation and test use disjoint seed sets; scenario files are hashed.
- **Conditions:** nominal; sensor-perturbed (P2 ranges); dynamics-perturbed (P3 ranges); both. On the first 50 test scenarios also: out-of-range (1.5× ranges at corners) and an unseen noise model (multiplicative range noise plus 1% spike outliers).
- **Stopping protocol (mirrors hardware):** deterministic actions; the wrapper stops the robot when the *estimated* goal distance falls below 0.20 m; success is then scored on ground truth.
- **Metrics:** success; collision; timeout; mis-stop (stopped by estimate but outside tolerance in ground truth); time to goal (successes); ground-truth path length; minimum clearance; safety interventions (SITL and hardware only); nominal-to-perturbed degradation in percentage points.
- **Statistics:** Wilson 95% intervals per cell; show per-seed points; paired comparisons on identical scenario and draw. Call an effect real only if it has the same sign in every seed and the pooled paired interval excludes zero. Also report cost of robustness (P4 vs P1 nominal) and interaction (P4 - P1 against the sum of P2 - P1 and P3 - P1).
- **Mechanism:** one-factor sweeps on trained policies (noise, dropout, latency, offset, odometry scale, gain, lag, delay, friction); no retraining.
- **Compute:** the full matrix with 3 seeds is about 6,000 episodes, roughly one training run's worth of steps. Halve the test scenarios if needed.
- **Baselines:** hand-coded controller on the same matrix; stationary and random policies for reference; Nav2 optional and labelled privileged.

9. Pre-training gate (G2): all must pass before PPO runs

#	Check	Pass criterion
1	SB3 check_env	Passes; observations float32, in bounds, no NaN or inf
2	Scenario generator	Same seed gives same layout; train/val/test seeds disjoint
3	Reset soak	1,000 consecutive resets: pose within 2 cm and 1°, zero velocity, fresh scan, correct goal geometry, no overlaps, zero failures
4	Timing	Every step uses a strictly newer scan; control, scan and hold periods match config; sim time used consistently
5	Preprocessing parity	Gym and deployment modules give identical observations on the same recorded scans (tolerance 1e-6)
6	Termination	Wall hit gives terminated and collision; goal gives terminated and success; timeout gives truncated only; collision triggers within 1 cm of footprint contact
7	Reward audit	Stationary, random, spin and hand-coded traces behave as intended
8	Hand-coded controller	Target at least 90% success on nominal scenarios (heuristic); no collapse at randomization corners
9	Soak and recovery	100k random steps with the chosen worker count, no crash, memory growth under 5%; kill-and-resume works
10	PPO smoke test	Learns empty-arena goal reaching; if not, suspect a bug, not hyperparameters
11	After randomizers	Isolation tests (Section 6) pass; a short P4 pilot still learns

10. Seven-day simulation plan

[S] = Saigirish, [V] = Siddhi. Training runs unattended from Day 4 evening; the plan has little slack, so the cut order below is agreed in advance.

Day	Tasks	Exit / gate
1	**Setup and audit.** [S] Pin environment (Docker or venv), install TB3 simulation, SB3, Gymnasium; record versions. Run the Burger headless; inspect /scan, /odom, /cmd_vel, /clock, QoS and LaserScan metadata. Single-worker throughput benchmark. [V] Confirm hardware availability and LiDAR model; create repo skeleton; define scenario-file format and logging schema.	G0: hardware yes/no. G1: throughput known, budget computed
2	**Environment v1.** [S] Gym env: shared preprocessing module, estimated-pose model, action mapping, obstacle pool and reset, ground-truth collision and goal, terminated/truncated, reward components, motion-distortion wrapper. [V] Hand-coded controller, scenario generator (train/val/test/stress seeds), metrics logger.	10k random steps without crash
3	**Gate G2.** [S+V] Run checks 1-10 of Section 9 (start the reset soak early; run it overnight if needed). Benchmark 1/2/4 workers. [S] Launch the P1 pilot (about 200-300k steps, 1 seed) once checks pass.	Checklist passes or fixes documented
4	**Freeze and randomize.** [S] Review the pilot: success curve, clearance, action jitter vs controller; decide clearance and smoothness terms; freeze reward, observation and hyperparameters (G3). Implement P2 and P3 randomizers. [V] Isolation tests and corner-case controller runs; shrink ranges if needed; freeze ranges. Short P4 pilot. Evening: launch the seed-major queue.	G3: design frozen; queue running
5	**Train and build the harness.** [S] Monitor training, fix crashes, export a seed-0 policy to ONNX or numpy with a parity test. [V] Evaluation harness: fixed scenario files, paired perturbation draws, matrix runner, Wilson intervals, plots; run the hand-coded baseline over the matrix (light CPU use).	Harness validated on the baseline
6	**Pilot evaluation and SITL.** [V] ROS 2 deployment node skeleton using the shared preprocessing; run the exported policy inside Gazebo and compare with the Gym trajectories (software in the loop). [S] Sweep scripts; run a small matrix on seed-0 policies to validate the harness end to end. **End of Day 6 decision:** if seed 2 cannot finish by Day 7 noon, drop it and document.	Seed 0 evaluated on a subset; cut decision made
7	**Full evaluation and handoff.** Run the full matrix for completed seeds, out-of-range and unseen-noise tests, stress set and sweeps. Produce tables and plots. Write results and limitations. Sign off the hardware-readiness checklist below.	Simulation deliverable complete

Cut order if time runs short

1) seed 2; 2) test scenarios 100 to 50; 3) stress set and sweep breadth; 4) steps per run (equally, not below about 500k); 5) SITL moved to the hardware phase. **Never cut:** the Section 9 gate, factor-isolation tests, equal budgets, the held-out test set, the hand-coded baseline on at least the nominal condition.

Hardware-readiness checklist (end of Day 7)

- Policy files for the pre-registered seed of each policy (seed 0 or the median by validation), each with a passing parity test.
- SITL pass: the deployment node reproduces Gym behaviour in Gazebo for identical initial conditions.
- Interface contract list to verify on the robot: LiDAR model and LaserScan fields, scan QoS and rate, `/cmd_vel` message type and timeout behaviour, frames, odometry convention.
- Safety layer specification, fixed before trials: velocity clamps (0.15 m/s, 0.6 rad/s), stale-scan watchdog (about two scan periods), deadman switch, teleoperation override, physical kill switch. Interventions count as failures; thresholds are never tuned after seeing results.
- Trial plan: external ground truth (overhead camera or measured tape grid), about 25-30 trials per policy, interleaved order, matched scenarios, logged battery voltage. One deployed seed per policy measures episode variance only, not training variance.

11. Repository and reproducibility conventions

Layout: `configs/` (env, ppo, rand_p2, rand_p3, scenarios), `envs/` (nav_env, preprocessing, estimated_pose, scan_model, dynamics_model, gazebo_interface), `baselines/`, `train/`, `eval/`, `analysis/`, `deploy/` (ROS 2 package), `tests/`, `runs/`.

- One YAML per run; log the git hash and config hash. Pin ROS, Gazebo, Python and package versions. Gazebo Classic is end-of-life (January 2025), so keep the installer packages.
- Independent random-number streams (for example `numpy.random.SeedSequence.spawn`) for layout, task, sensor noise, dynamics and PPO, so toggling randomization never changes the layout sequence.
- Gazebo is not bit-reproducible: log every layout and randomization draw per episode (scenario ID, sampled parameters, outcome, ground-truth path length, minimum clearance, estimated vs true final distance).

12. Known unknowns to resolve early

Unknown	How to resolve	When
Installed LiDAR (LDS-01 or LDS-02) and the real LaserScan layout	Inspect hardware and `ros2 topic echo /scan`; update config	Day 1
Simulated scan rate and beam count vs the real sensor	Echo the simulated /scan; match rate in the Gazebo model	Day 1
Gazebo throughput and worker scaling on the laptop	Benchmark 1/2/4 workers	Days 1 and 3
Whether SB3 bootstraps truncated episodes in the pinned version	Unit test	Day 2-3
Real `/cmd_vel` type and command timeout	Check on the robot	Hardware phase
Whether the provisional ranges are physically meaningful	Characterize the real robot; otherwise label provisional in all results	Hardware phase

13. References for theory and implementation

Topic	Link
PPO (paper)	https://arxiv.org/abs/1707.06347
PPO explained (OpenAI Spinning Up)	https://spinningup.openai.com/en/latest/algorithms/ppo.html
Stable-Baselines3 PPO docs	https://stable-baselines3.readthedocs.io/en/master/modules/ppo.html
SB3 custom environments and tips	https://stable-baselines3.readthedocs.io/en/master/guide/custom_env.html https://stable-baselines3.readthedocs.io/en/master/guide/rl_tips.html
SB3 callbacks and vectorized envs	https://stable-baselines3.readthedocs.io/en/master/guide/callbacks.html https://stable-baselines3.readthedocs.io/en/master/guide/vec_envs.html
SB3 paper (JMLR 2021)	https://jmlr.org/papers/v22/20-1364.html
What matters in on-policy RL (implementation details)	https://arxiv.org/abs/2006.05990
Gymnasium: terminated vs truncated (migration guide)	https://gymnasium.farama.org/introduction/migration_guide/
Gymnasium: creating a custom environment	https://gymnasium.farama.org/introduction/create_custom_env/
Domain randomization for visual sim-to-real (Tobin et al.)	https://arxiv.org/abs/1703.06907
Dynamics randomization (Peng et al.)	https://arxiv.org/abs/1710.06537
Sim-to-real: actuator models and latency (Tan et al.)	https://arxiv.org/abs/1804.10332
Sim-to-real in deep RL for robotics: survey (Zhao et al.)	https://arxiv.org/abs/2009.13303
Mapless navigation from range data with deep RL (Tai et al.)	https://arxiv.org/abs/1703.00420
Seeds, reproducibility (Henderson et al.)	https://arxiv.org/abs/1709.06560
Reliable evaluation with few runs (Agarwal et al.) and rliable	https://arxiv.org/abs/2108.13264 https://github.com/google-research/rliable
Wilson score interval	https://en.wikipedia.org/wiki/Binomial_proportion_confidence_interval
TurtleBot3 simulation and features	https://emanual.robotis.com/docs/en/platform/turtlebot3/simulation/ https://emanual.robotis.com/docs/en/platform/turtlebot3/features/
LiDAR specifications (LDS-01, LDS-02)	https://emanual.robotis.com/docs/en/platform/turtlebot3/appendix_lds_01/ https://emanual.robotis.com/docs/en/platform/turtlebot3/appendix_lds_02/
Gazebo Classic entity-state services and sim time (ROS 2)	https://github.com/ros-simulation/gazebo_ros_pkgs/wiki/ROS-2-Migration:-Entity-states https://github.com/ros-simulation/gazebo_ros_pkgs/wiki/ROS-2-Migration:-ROS-Clocks-and-sim-time https://docs.ros.org/en/humble/p/gazebo_msgs/srv/SetEntityState.html
Gazebo Classic end-of-life	https://discourse.openrobotics.org/t/gazebo-classic-end-of-life/48748
ROS 2 Humble QoS settings	https://docs.ros.org/en/humble/Concepts/Intermediate/About-Quality-of-Service-Settings.html

Appendix: changes made to the source draft

The supplied draft was cut off partway through the PPO configuration table, so evaluation, deployment and timeline content was rebuilt. Main changes:

- Reward progress term changed to a plain ground-truth distance difference on every transition; the potential form with zero terminal potential rewards collisions far from the goal.
- Added concrete provisional values: arena, goal tolerance, timeout, randomization ranges, evaluation set sizes, hand-coded controller rule.
- Motion distortion fixed as shared nominal fidelity for all policies; final-checkpoint rule fixed; mis-stop metric added; P4 latency constraint added.
- PPO table completed (CPU device, torch threads, reward scaling, log std); compute budget rule and seed-major queue added, with a short P4 pilot before queueing.
- Timeline replaced by a 7-day simulation plan with gates, an agreed cut order and a hardware-readiness checklist.
- Kept from the draft: 2×2 design, wheel radius excluded from P3, estimated-pose model, ground-truth collision, truncation semantics, previous-action input, stopping protocol mirroring hardware.